

EXTENDS *FiniteSets, Sequences, Naturals, TLC*

Model of a Consistent *S3* Object Store. Author: *Steve Loughran*

* Licensed to the *Apache* Software Foundation (*ASF*) under one
 * or more contributor license agreements. See the NOTICE file
 * distributed with this work for additional information
 * regarding copyright ownership. The *ASF* licenses this file
 * to you under the *Apache* License, Version 2.0 (the
 * “License”); you may not use this file except in compliance
 * with the License. You may obtain a copy of the License at
 *
 * <http://www.apache.org/licenses/LICENSE-2.0>
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an “AS IS” BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”,
 “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are
 to be interpreted as described in *RFC* 2119.

A Path is a list of Path elements which represents a path to a file, directory or symbolic link
 Path elements are non-empty strings. The exact set of valid strings MAY be specific to a particular
FileSystem implementation.
 Path elements MUST NOT be in { “”, “.”, “..”, “/” }
 Path elements MUST NOT contain the characters ‘{ ‘/’, ‘:’ }’.
 Filesystems MAY have other strings that are not permitted in a path element.
 When validating path elements, the exception ‘InvalidPathException’ SHOULD
 be raised when a path is invalid [HDFS]

CONSTANTS

<i>Paths</i> ,	<i>the non – finite set of all possible valid paths</i>
<i>PathsAndRoot</i> ,	<i>Paths and the “root” path; the latter is read – only</i>
<i>Data</i> ,	<i>the non – finite set of all possible sequences of bytes</i>

Timestamp, *A timestamp*
Byte,
NonEmptyString

ASSUME *NonEmptyString* $\in$ (STRING $\setminus$ "")

ASSUME *PathsAndRoot* $\in$ STRING
 ASSUME *Paths* $\in$ (*PathsAndRoot* $\setminus$ "")

Timestamps are positive integers since the epoch.
 ASSUME *Timestamp* $\in$ Nat $\wedge$ *Timestamp* > 0

Byte type
 ASSUME *Byte* $\in 0 \dots 255$

Data is a sequence of bytes
 ASSUME *Data* $\in$ Seq(*Byte*)

There is a predicate to validate a pathname.
This is considered implementation – specific.
It could be describable as a regular expression specific to each implementation,
though constraints such as “no two adjacent '/' characters” might make for a complex regexp.
Perhaps each FS would have a set of regexps which all must be valid for
a path to be considered valid.
 CONSTANT *is_valid_pathname*(_)

All paths can be evaluated to see if their pathname is valid
 ASSUME $\forall p \in \text{Paths} : \text{is_valid_pathname}(p) \in \text{BOOLEAN}$

This specification defines

- * A model of a consistent object store : a consistent store of data and metadata, one without any notion of a “directory hierarchy”. It is intended to model object stores such as Amazon S3, and includes its multipart PUT API.*
- * An API for communicating with object stores from Hadoop filesystems.*

It is intended to be a foundation for defining algorithms with worth with S3, such as the s3guard commit algorithm.

CONSTANTS

MetadataKeys, the set of all possible metadata keys
MetadataValues, the non – finite set of all possible metadata values
Etag,
MultipartPutId,
PartId

There are some metadata keys which are system metadata entries.
Those MAY be queried but SHALL NOT be explicitly set.(more specifically, they ll be ignored if you try.

ASSUME *MetadataKeys* $\in$ *NonEmptyString*

ASSUME *MetadataValues* $\in$ STRING

ASSUME *Etag* $\in$ *NonEmptyString*

ASSUME *MultipartPutId* $\in$ *NonEmptyString*

Only 11,000 parts are allowed

ASSUME *PartId* $\in$ 1 .. 11000

CONSTANT *is_valid_metadata_key*(-)

All *metadata* keys can be evaluated for validity

ASSUME $\forall e \in \text{MetadataKeys} : \text{is_valid_metadata_key}(e) \in \text{BOOLEAN}$

The patch matching algorithm used in the list operation

CONSTANT *path_matches*(-, -, -)

This should really be defined by looking inside the strings.

It is: all paths starting with the prefix up to those ending in the suffix

ASSUME $\forall p \in \text{Paths}, \text{prefix}, \text{delimiter} \in \text{Paths} : \text{path_matches}(p, \text{prefix}, \text{delimiter}) \in \text{BOOLEAN}$

A function to return an *etag* of some data

CONSTANT *etag_of*(-)

A function to return an *etag* of a multipart operation; implementation specific

CONSTANT *etag_of_multipart_operation*(-)

Etags are strings, hence in *MetadataValues*.

ASSUME $\forall d \in \text{Data} : \text{etag_of}(d) \in \text{Etag}$

This is commented out as it is not a requirement that etags are the same for an equivalent sequence of bytes. All that matters is that one is generated. ASSUME $\forall d, e \in \text{Data} : d = e \Rightarrow \text{etag_of}(d) = \text{etag_of}(e) \in \text{STRING}$

VARIABLE *store* The object store
 VARIABLE *pending* Pending requests

Exception logic

BadRequest $\triangleq$ “BadRequest”
NotFound $\triangleq$ “NotFound”
Success $\triangleq$ “Success”

MetadataEntry $\triangleq$ [
 key : *MetadataKeys*, The key of the entry
 value : *MetadataValues* the value of this *metadata* entry
]

SystemMetadata $\triangleq$ [
 size : *Nat*,
 created : *Timestamp*
]

A store : *path* $\rightarrow$ (data, user-md, system-md)
 update: *PUT*, *DELETE* query: *GET*, *HEAD*, *LIST*(*path*)

StoreEntry $\triangleq$ [
 data : *Data*, the data in the entry
 created : *Timestamp*, timestamp
 etag : *MetadataValues*
]

ListingEntry $\triangleq$ [
 path : *Paths*, The path to the entry
 data : *Data*, the data in the entry
 created : *Timestamp*, timestamp
 etag : *MetadataValues*,
 metadata : *MetadataEntry* it’s a set
]

The check for a path having an entry is pulled out for declaring invariants
has_entry(*s*, *p*) $\triangleq$ *p* $\in$ DOMAIN *s*

$$\text{PendingMultipartPartRequest} \triangleq [$$

$$\text{putId} : \text{MultipartPutId},$$

$$\text{part} : \text{PartId},$$

$$\text{data} : \text{Data}$$

$$]$$

$$\text{PendingMultipartPartResponse} \triangleq [$$

$$\text{etag} : \text{Etag}$$

$$]$$

$$\text{PendingMultipartPutPart} \triangleq [$$

$$\text{data} : \text{Data},$$

$$\text{etag} : \text{Etag}$$

$$]$$

A pending *Multipart* Upload has an *ID* and start time, which is used to define the final create time of the committed operation

$$\text{PendingMultipartOperation} \triangleq [$$

$$\text{id} : \text{STRING},$$

$$\text{path} : \text{Paths},$$

$$\text{started} : \text{Timestamp},$$

$$\text{parts} : [\text{PartId} \rightarrow \text{PendingMultipartPutPart}]$$

$$]$$

Actions. Note how some post conditions are explicitly called out. They are superfluous, in the model, but they do declare final state for testability

PUT: update the store with the newly uploaded data. This definition is consistent: the store changes are immediately visible, even if there was an existing entry.

$$\text{doPut}(\text{path}, \text{data}, \text{current_time}, \text{result}) \triangleq$$

$$\text{LET } \text{validArgs} \triangleq \text{path} \in \text{Paths} \wedge \text{data} \in \text{Data} \wedge \text{current_time} \in \text{Timestamp}$$

$$\text{IN}$$

$$\vee \wedge \neg \text{validArgs}$$

$$\wedge \text{result}' = \text{BadRequest}$$

$$\wedge \text{UNCHANGED } \langle \text{store}, \text{pending} \rangle$$

$$\vee \wedge \text{validArgs}$$

$$\wedge \text{result}' = \text{Success}$$

$$\wedge \text{UNCHANGED } \text{pending}$$

$$\wedge \text{store}' = [\text{store} \text{ EXCEPT } ![\text{path}] = [\text{data} \mapsto \text{data}, \text{created} \mapsto \text{current_time}, \text{etag} \mapsto \text{etag_of}(\text{data})]]$$

GET: $\text{path} \rightarrow \text{data}$ as well as summary *metadata*

$$\begin{aligned}
& doGet(path, metadata, data, current_time, result) \triangleq \\
& \text{LET} \\
& \quad validArgs \triangleq path \in PathsAndRoot \\
& \quad exists \triangleq has_entry(store, path) \\
& \quad entry \triangleq store[path] \\
& \text{IN} \\
& \quad \vee \wedge \neg validArgs \\
& \quad \quad \wedge result' = BadRequest \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle \\
& \\
& \quad \vee \wedge validArgs \\
& \quad \quad \wedge path = "" \\
& \quad \quad \wedge result' = Success \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle \\
& \quad \quad \wedge data' = \{\} \\
& \\
& \quad \vee \wedge validArgs \\
& \quad \quad \wedge \neg exists \\
& \quad \quad \wedge result' = NotFound \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle \\
& \\
& \quad \vee \wedge validArgs \\
& \quad \quad \wedge exists \\
& \quad \quad \wedge result' = Success \\
& \quad \quad \wedge data' = store[path].data \\
& \quad \quad \wedge metadata' = [created \mapsto entry.created, length \mapsto Len(entry.data), etag \mapsto entry.etag] \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle
\end{aligned}$$

HEAD: the metadata without the data

$$\begin{aligned}
& doHead(path, metadata, current_time, result) \triangleq \\
& \text{LET} \\
& \quad validArgs \triangleq path \in PathsAndRoot \\
& \quad exists \triangleq has_entry(store, path) \\
& \quad entry \triangleq store[path] \\
& \text{IN} \\
& \quad \vee \wedge \neg validArgs \\
& \quad \quad \wedge result' = BadRequest \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle \\
& \\
& \quad \vee \wedge validArgs \\
& \quad \quad \wedge path = "" \\
& \quad \quad \wedge result' = Success \\
& \quad \quad \wedge metadata' = [created \mapsto 0, length \mapsto 0] \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle \\
& \\
& \quad \vee \wedge validArgs
\end{aligned}$$

$$\begin{aligned}
& \wedge \neg exists \\
& \wedge result' = NotFound \\
& \wedge UNCHANGED \langle store, pending \rangle \\
\vee & \wedge validArgs \\
& \wedge exists \\
& \wedge result' = Success \\
& \wedge metadata' = [created \mapsto entry.created, length \mapsto Len(entry.data), etag \mapsto entry.etag] \\
& \wedge UNCHANGED \langle store, pending \rangle \\
doDelete(path, current_time, result) & \triangleq \\
\text{LET} & \\
& validArgs \triangleq path \in Paths \\
& exists \triangleq has_entry(store, path) \\
\text{IN} & \\
\vee & \wedge \neg validArgs \\
& \wedge result' = BadRequest \\
& \wedge UNCHANGED \langle store, pending \rangle \\
\vee & \wedge validArgs \\
& \wedge \neg exists \\
& \wedge result' = NotFound \\
& \wedge UNCHANGED \langle store, pending \rangle \\
\vee & \wedge validArgs \\
& \wedge exists \\
& \wedge result' = Success \\
& \wedge store' = [p \in (\text{DOMAIN } store \setminus path) \mapsto store[p]] \\
& \wedge UNCHANGED pending \\
doCopy(source, dest, current_time, result) & \triangleq \\
\text{LET} & \\
& validArgs \triangleq source \in Paths \wedge dest \in Paths \wedge current_time \in Timestamp \\
& exists \triangleq has_entry(store, source) \\
\text{IN} & \\
\vee & \wedge \neg validArgs \\
& \wedge result' = BadRequest \\
& \wedge UNCHANGED \langle store, pending \rangle \\
\vee & \wedge validArgs \\
& \wedge \neg exists \\
& \wedge result' = NotFound \\
& \wedge UNCHANGED \langle store, pending \rangle \\
\vee & \wedge validArgs \\
& \wedge exists
\end{aligned}$$

$$\begin{aligned}
& \wedge result' = Success \\
& \wedge store' = [store \text{ EXCEPT } ![dest] = [data \mapsto store[source].data, created \mapsto current_time]] \\
& \wedge UNCHANGED \ pending
\end{aligned}$$

The list operation returns the *metadata* of all entries in the object store whose path matches the prefix/suffix pattern.

S3 also returns a string sequence of common subpath underneath, essential “what look like directories”

$$pathsMatchingPrefix(prefix, suffix) \triangleq \forall path \in \text{DOMAIN } store : path_matches(path, prefix, suffix)$$

$$doList(prefix, suffix, current_time, result, listing) \triangleq$$

LET

$$validArgs \triangleq prefix \in \text{STRING} \wedge suffix \in \text{STRING}$$

IN

$$\begin{aligned}
& \vee \wedge \neg validArgs \\
& \wedge result' = BadRequest \\
& \wedge UNCHANGED \langle store, pending \rangle \\
\\
& \vee \wedge validArgs \\
& \wedge result' = Success \\
& \wedge listing' = [path \in pathsMatchingPrefix(prefix, suffix) \mapsto \\
& \quad [path \mapsto path, \\
& \quad \quad created \mapsto store[path].created, \\
& \quad \quad length \mapsto Len(store[path].data), \\
& \quad \quad etag \mapsto store[path].etag]] \\
& \wedge UNCHANGED \langle store, pending \rangle
\end{aligned}$$

Initiate a multipart *PUT*. The destination is specified; the create time of the final artifact is set to the current server time. A unique *ID* is returned. There is no requirement for the destination to be unique: multiple requests may target the same destination, with the order of the commit operation defining the order in which the results become visible.

$$doInitiateMultipartPut(dest, current_time, result, operationId) \triangleq$$

LET

$$validArgs \triangleq dest \in Paths \wedge current_time \in Timestamp$$

$$newPartId \triangleq \text{CHOOSE } id \in MultipartPutId : \neg id \in \text{DOMAIN } pending$$

IN

$$\begin{aligned}
& \vee \wedge \neg validArgs \\
& \wedge result' = BadRequest \\
& \wedge UNCHANGED \langle store, pending \rangle \\
\\
& \vee \wedge validArgs \\
& \wedge result' = Success \\
& \wedge UNCHANGED store \\
& \wedge operationId' = newPartId \\
& \wedge pending' = [pending \text{ EXCEPT } ![newPartId] = \\
& \quad [path \mapsto dest, created \mapsto current_time]]
\end{aligned}$$

PUT a single part for an operation. Result is an *etag* to be used for the ordered sequence

$$\begin{aligned}
& doPutPart(operationId, partId, part_data, current_time, result, etagResult) \triangleq \\
& \text{LET} \\
& \quad validArgs \triangleq operationId \in \text{DOMAIN } pending \wedge part_data \in Data \wedge partId \in PartId \\
& \quad etagVal \triangleq etag_of(part_data) \\
& \text{IN} \\
& \quad \vee \wedge \neg validArgs \\
& \quad \quad \wedge result' = BadRequest \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle \\
& \quad \vee \wedge validArgs \\
& \quad \quad \wedge result' = etagVal \\
& \quad \quad \wedge etagResult' = etagVal \\
& \quad \quad \wedge \text{UNCHANGED } store \\
& \quad \quad \wedge pending' = [pending \text{ EXCEPT} \\
& \quad \quad \quad ![operationId] = [\\
& \quad \quad \quad \quad path \mapsto pending[operationId].dest, \\
& \quad \quad \quad \quad parts \mapsto [pending[operationId].parts \text{ EXCEPT} \\
& \quad \quad \quad \quad \quad ![partId] = [data \mapsto part_data, etag \mapsto etagVal]] \\
& \quad \quad \quad] \\
& \quad \quad]
\end{aligned}$$

The commit operation is the most complex. The part list supplied defines the order in which the supplied parts are saved to the store.

TODO: work out how to declare that all data is the ordered appending of the data of the list of parts. Recurse?

$$\backslash * alldata \triangleq \forall [part \in (1 \dots Len(parts) - 1)] \text{ Append}(upload[parts[part]], upload[parts[part + 1]])$$

doCommitMultipartPut(*operationId*, *parts*, *current_time*, *result*) $\triangleq$

$$\begin{aligned}
& \text{LET} \\
& \quad upload \triangleq pending[operationId] \\
& \quad validArgs \triangleq (operationId \in \text{DOMAIN } pending) \wedge (parts \in Seq(PartId)) \\
& \quad \quad \wedge (\forall p \in parts : p \in \text{DOMAIN } upload.parts) \wedge (\forall p \in \text{DOMAIN } upload.parts : p \in parts) \\
& \quad \quad alldata \triangleq parts \\
& \quad \quad etag \triangleq etag_of_multipart_operation(upload) \\
& \text{IN} \\
& \quad \vee \wedge \neg validArgs \\
& \quad \quad \wedge result' = BadRequest \\
& \quad \quad \wedge \text{UNCHANGED } \langle store, pending \rangle \\
& \quad \vee \wedge validArgs \\
& \quad \quad \wedge result' = Success \\
& \quad \quad \wedge pending' = [p \in (\text{DOMAIN } pending \setminus operationId) \mapsto pending[p]] \\
& \quad \quad \wedge store' = [store \text{ EXCEPT } ![upload.path] = [data \mapsto alldata, created \mapsto upload.created, etag \mapsto etag]]
\end{aligned}$$

Abort the multipart put operation. All pending data is deleted; the pending operation record removed.

$doAbortMultipartPut(operationId, current_time, result) \triangleq$

LET

$validArgs \triangleq operationId \in \text{DOMAIN } pending$

IN

$\vee \wedge \neg validArgs$

$\wedge result' = BadRequest$

$\wedge \text{UNCHANGED } \langle store, pending \rangle$

$\vee \wedge validArgs$

$\wedge result' = Success$

$\wedge \text{UNCHANGED } store$

$\wedge pending' = [p \in (\text{DOMAIN } pending \setminus operationId) \mapsto pending[p]]$

$PutInvariant \triangleq \forall p \text{ in } Paths: doDelete(p, Success) \Rightarrow \neg has_entry(store', p)$

$DeleteInvariant \triangleq \forall p \text{ in } Paths: doDelete(p, Success) \Rightarrow \neg has_entry(store', p)$

now define action messages which can be queued for processing; we consider them to be processed in a serial order

Action Records

$putAction \triangleq [$
 $verb : "PUT",$
 $path : \text{STRING},$
 $data : [Nat \rightarrow Nat]$
 $]$

$deleteAction \triangleq [$
 $verb : "DELETE",$
 $path : \text{STRING}$
 $]$

$getAction \triangleq [$
 $verb : "GET",$
 $path : \text{STRING},$
 $data : \text{STRING}$
 $]$

```

headAction  $\triangleq$  [
  verb : "HEAD",
  path : STRING
]

copyAction  $\triangleq$  [
  verb : "COPY",
  source : STRING,
  dest : STRING
]

listAction  $\triangleq$  [
  verb : "LIST",
  prefix : STRING,
  delimiter : STRING
]

```

Process a request, generate a result.

```

process(request, result, metadata, body, current_time)  $\triangleq$ 
  LET verb  $\triangleq$  request.verb
  IN
     $\vee$  verb = "PUT"       $\wedge$  doPut(request.path, request.data, current_time, result)
     $\vee$  verb = "GET"       $\wedge$  doGet(request.path, metadata, body, current_time, result)
     $\vee$  verb = "HEAD"      $\wedge$  doHead(request.path, metadata, current_time, result)
     $\vee$  verb = "DELETE"    $\wedge$  doDelete(request.path, current_time, result)
     $\vee$  verb = "COPY"      $\wedge$  doCopy(request.source, request.dest, current_time, result)
     $\vee$  verb = "LIST"      $\wedge$  doList(request.prefix, request.suffix, current_time, result, body)

```

The store state invariant not only declares the type of the store, it declares attributes of the *has_entry* operator which are superfluous given the definition of *has_entry()* as the path being in the domain of the store. It's explicit for those implementors planning to write tests.

```

StoreStateInvariant  $\triangleq$ 
   $\wedge$  store  $\in$  [Paths  $\rightarrow$  StoreEntry]
   $\wedge$  pending  $\in$  [MultipartPutId  $\rightarrow$  PendingMultipartOperation]

```

The initial state of the store is that it is empty.

Notice how this ignores the root entry, "".

This is a special entry: object stores are not filesystems: there is no root node equivalent to "/"

```

InitialStoreState  $\triangleq$ 
   $\wedge$  StoreStateInvariant
   $\wedge$  DOMAIN store = {}
   $\wedge$  DOMAIN pending = {}

```

The amount of data you get back is the amount of data you are told comes back.

GetLengthInvariant $\triangleq$
 $\forall path \in Paths, sysMd \in SystemMetadata, data \in Data, t \in Timestamp :$
 $doGet(path, sysMd, data, t, Success) \triangleq > Len(data) \triangleq sysMd.length$

THEOREM $\forall path \in Paths, sysMd \in SystemMetadata, data \in Data, t \in Timestamp :$
 TRUE

The *metadata* that comes from a *doHead()* MUST match that from a *doGet()*

See: *HADOOP-11202 SequenceFile* crashes with encrypted files that are shorter than *FileSystem.getStatus(path)*

THEOREM *GetAndHeadInvariant* $\Rightarrow$
 $\forall path \in Path, sysMd \in SystemMetadata, data \in Data, t \in Timestamp :$
 $doGet(path, sysMd, data, t, Success) \triangleq > doHead(path, sysMd, t, Success)$

The details you get back in a listing match the details you get back from a *doGet* call on the specific path of course, on an eventually consistent object store, there may be lag

TODO: ListAndGetInvariant The subset of *metadata* returned from a listing matches that of the individual records returned in *HEAD* and *GET*.

THEOREM *InitialStoreState* $\Rightarrow \Box StoreStateInvariant$

\ * Modification History
 \ * Last modified Sun Dec 10 21:53:18 GMT 2017 by stevel
 \ * Created Sun Jun 19 18:07:44 BST 2016 by stevel